



ICT Training Center

Il tuo partner per la Formazione e la Trasformazione digitale della tua azienda



SPRING AI

GENERATIVE ARTIFICIAL INTELLIGENCE CON JAVA

Simone Scannapieco

Corso avanzato per Venis S.p.A, Venezia, Italia

Novembre 2025



TEXT-TO-SPEECH

- ➔ Sintetizzazione sonora a partire da testo (opposto della trascrizione)
- ⚠ Non possibile utilizzare le capacità multimodali dei LLM generalisti
- ➔ Compatibilità Spring AI (al 11/2025)
 - ➔ OpenAI API
 - ➔ ElevenLabs
- ➔ Possibilità di scelta fra tono, voci, velocità, ...
- ➔ Ambiti applicativi
 - ➔ IVR (Interactive Voice Response) - Sistema telefonico automatizzato di seconda generazione
 - ➔ Tecnologie assistive dedicate alle diverse abilità
 - ➔ ...

Interfaccia di base

```
public interface TextToSpeechModel extends Model<TextToSpeechPrompt, TextToSpeechResponse>,
    StreamingTextToSpeechModel {

    default byte[] call(String text) {
        TextToSpeechPrompt prompt = new TextToSpeechPrompt(text);
        ModelResult<byte[]> result = call(prompt).getResult();
        if (result == null) {
            return new byte[0];
        }
        byte[] output = result.getOutput();
        return (output != null) ? output : new byte[0];
    }

    @Override
    TextToSpeechResponse call(TextToSpeechPrompt prompt);

    default TextToSpeechOptions getDefaultOptions() {
        return TextToSpeechOptions.builder().build();
    }
}
```

- ⚠ *Breaking change* al passaggio verso Spring AI 1.1.0
- ⚠ Transizione verso entità funzionali e *major refactoring*
- ⚠ Consultare la documentazione costantemente

Controller di sintetizzazione audio

```
@RestController
@RequestMapping("/api")
public class AudioController {

    private final TextToSpeechModel speechModel;

    AudioController(TextToSpeechModel speechModel) {
        this.speechModel = speechModel;
    }

    @GetMapping("/speech")
    String speech(@RequestParam("message") String message) throws IOException {
        byte[] audioBytes = speechModel.call(message);
        Path path = Paths.get("output.mp3");
        Files.write(path, audioBytes);
        return "MP3 salvato con successo in " + path.toAbsolutePath();
    }
}
```

- ➔ Impostazioni di *default*
 - ➔ GPT 4.0 mini TTS
 - ➔ *Response format* in JSON
 - ➔ Velocità di sintetizzazione 1.0
 - ➔ Voce Alloy
- ➔ Istanziato un bean `OpenAiAudioSpeechModel`

Controller di sintetizzazione audio

```
@RestController
@RequestMapping("/api")
public class AudioController {

    ...

    @GetMapping("/speech-options")
    String speechWithOptions(@RequestParam("message") String message) throws IOException {
        TextToSpeechResponse speechResponse = speechModel.call(
            new TextToSpeechPrompt(message,
                OpenAiAudioSpeechOptions.builder()
                    .voice(OpenAiAudioApi.SpeechRequest.Voice.NOVA)
                    .speed(2.0f)
                    .responseFormat(OpenAiAudioApi.SpeechRequest.AudioResponseFormat.MP3)
                    .build());
        Path path = Paths.get("speech-options.mp3");
        Files.write(path, speechResponse.getResult().getOutput());
        return "MP3 salvato con successo in " + path.toAbsolutePath();
    }
}
```

- ➔ Popolato un `TextToSpeechPrompt` con tutte le sovrascritture volute rispetto al *default*
- ➔ `speechResponse.getResult().getOutput()` ripulisce l'output da tutti i metadati restituendo il dato grezzo (`byte[]`)

Configurazione applicativo

```
spring:
  application:
    name: demo
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      audio:
        speech:
          model: gpt-4o-mini-tts # gpt-4o-mini-tts (opt. speed and costs)
                                # gpt-4o-tts (higher quality)
                                # tts-1 (opt. speed)
                                # tts-1-hd (opt. quality)
          voice: alloy           # alloy, echo, fable, onyx, nova, shimmer
          response-format: mp3  # mp3, opus, aac, flac, wav, pcm
          speed: 1.0            # [0.25, 4.0]
          ...
```


- ⚠ Impossibile utilizzare compatibilità per LLM generalista
- ⚠ Obbligatorio *workaround* di *call* diretta API del AI provider tramite Spring Web

Approccio REST per sintetizzazione vocale: Gemini - singolo

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-preview-tts:generateContent" \  
-H "x-goog-api-key: $GEMINI_API_KEY" \  
-X POST \  
-H "Content-Type: application/json" \  
-d '{  
  "contents": [{  
    "parts": [{  
      "text": "Say cheerfully: Have a wonderful day!"  
    }]  
  }],  
  "generationConfig": {  
    "responseModalities": ["AUDIO"],  
    "speechConfig": {  
      "voiceConfig": {  
        "prebuiltVoiceConfig": {  
          "voiceName": "Kore"  
        }  
      }  
    }  
  },  
  "model": "gemini-2.5-flash-preview-tts",  
}' | jq -r '.candidates[0].content.parts[0].inlineData.data' | \  
base64 --decode > out.pcm  
  
ffmpeg -f s16le -ar 24000 -ac 1 -i out.pcm out.wav
```

Approccio REST per sintetizzazione vocale: Gemini - conversazione

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-preview-tts:generateContent" \
-H "x-goog-api-key: $GEMINI_API_KEY" -X POST -H "Content-Type: application/json" \
-d '{
  "contents": [{
    "parts": [{
      "text": "Mario: Come va oggi Maria? Maria: Abbastanza bene, te?"
    }]
  }],
  "generationConfig": {
    "responseModalities": ["AUDIO"],
    "speechConfig": {
      "multiSpeakerVoiceConfig": {
        "speakerVoiceConfigs": [{
          "speaker": "Joe",
          "voiceConfig": {
            "prebuiltVoiceConfig": {
              "voiceName": "Kore"
            }
          }
        }, {
          "speaker": "Jane",
          "voiceConfig": {
            "prebuiltVoiceConfig": {
              "voiceName": "Puck"
            }
          }
        }
      ]
    }
  },
  "model": "gemini-2.5-flash-preview-tts",
  ' | jq -r '.candidates[0].content.parts[0].inlineData.data' | base64 --decode > out.pcm

ffmpeg -f s16le -ar 24000 -ac 1 -i out.pcm out.wav
```

➔ Servizio *text-to-speech* Gemini

- 1 Modifica dipendenze di progetto
- 2 Creazione modelli Gemini TTS *request* e *response*
- 3 Creazione modello per informazioni voci disponibili
- 4 Creazione convertitore *output* audio Gemini a formato riproducibile nel *web*
- 5 Modifica controllore MVC
- 6 Creazione pagina di presentazione
- 7 Test delle funzionalità su pagina di presentazione

Dipendenze applicativo

```
...  
<dependency>  
  <groupId>com.google.cloud</groupId>  
  <artifactId>google-cloud-texttospeech</artifactId>  
  <version>2.40.0</version>  
</dependency>  
...
```

Modello richiesta Gemini TTS

```
package it.venis.ai.spring.demo.model;

public class GeminiTtsRequest {
    private String text;
    private String voice;
    private String model;
    private String stylePrompt;

    ...
}
```

Modello risposta Gemini TTS

```
package it.venis.ai.spring.demo.model;

public class GeminiTtsResponse {
    private String audioBase64;
    private String mimeType;
    private String message;

    public GeminiTtsResponse(String audioBase64, String mimeType, String message) {
        this.audioBase64 = audioBase64;
        this.mimeType = mimeType;
        this.message = message;
    }

    ...
}
```

Modello informazioni voci disponibili

```
package it.venis.ai.spring.demo.model;

import java.util.List;

public class VoiceInfo {
    private String name;
    private List<String> languageCodes;
    private String gender;

    public VoiceInfo(String name, List<String> languageCodes, String gender) {
        this.name = name;
        this.languageCodes = languageCodes;
        this.gender = gender;
    }

    ...
}
```

Utilità convertitore PCM -> WAV

```
package it.venis.ai.spring.demo.util;

...

public class PcmToWavConverter {

    public static byte[] pcmToWav(byte[] pcmData, float sampleRate, int sampleSizeInBits,
                                   int channels, boolean signed, boolean bigEndian) throws IOException {

        // Crea la specifica del formato audio
        AudioFormat audioFormat = new AudioFormat(
            sampleRate,           // Frequenza di campionamento (Hz)
            sampleSizeInBits,     // Dimensione del campione in bit
            channels,             // Numero di canali
            signed,               // Con segno o senza segno
            bigEndian             // Big-endian o little-endian
        );

        // Racchiude i dati PCM in un ByteArrayInputStream
        ByteArrayInputStream pcmInputStream = new ByteArrayInputStream(pcmData);

        // Calcola la lunghezza del frame
        long frameLength = pcmData.length / audioFormat.getFrameSize();

        // Crea un AudioInputStream dai dati PCM
        AudioInputStream audioInputStream = new AudioInputStream(
            pcmInputStream,
            audioFormat,
            frameLength
        );

        ...
    }
}
```


Implementazione controllore REST

```
package it.venis.ai.spring.demo.controllers;
...
@RestController
@RequestMapping("/gemini/multi-modality")
public class GeminiMultimodalityController {

    /** Chiave API di Google AI recuperata dalle proprietà di configurazione */
    @Value("${GOOGLE_AI_API_KEY}")
    private String geminiApiKey;

    /** URL base dell'API Gemini per le richieste ai modelli */
    @Value("${gemini.api.models.url}")
    private String geminiApiUrl;

    /** Servizio per le operazioni multimodali (STT, ITT, TTS) */
    private final MultiModalityService multiModalityService;

    /** Client HTTP REST per le chiamate all'API di Gemini */
    private final RestTemplate restTemplate = new RestTemplate();

    /**
     * Costruttore del controller per la gestione delle funzionalità multimodali di Gemini.
     *
     * @param multiModalityService servizio per le operazioni multimodali (STT, ITT, TTS)
     */
    public GeminiMultimodalityController(MultiModalityService multiModalityService) {

        this.multiModalityService = multiModalityService;
    }
    ...
}
```

Pagina web

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gemini Text-to-Speech Demo</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      padding: 20px;
      display: flex;
      justify-content: center;
      align-items: center;
    }

    .container {
      background: white;
      border-radius: 20px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
      padding: 40px;
      max-width: 900px;
      width: 100%;
    }

    ...
  </body>
</html>
```

<https://github.com/simonescannapieco/spring-ai-advanced-dgroove-venis-code.git>
Branch: 18-spring-ai-gemini-ollama-multimodality-tts